CivicPulse — Advanced MERN Assessment Project

Sep 24, 2026 · @Sunain

Overview

Build **CivicPulse**, a full-stack MERN platform where citizens report neighborhood problems (broken roads, garbage piles, water leaks, broken streetlights, open manholes) and local authorities track, assign and resolve them in public.

The problem. In most cities, civic complaints go to phone lines or social media posts that disappear. Citizens never learn whether anyone acted, the same pothole gets reported 50 times, and authorities have no data on which areas are neglected or how long fixes take.

What the candidate builds. A web app with four parts:

1. **Citizen portal** — report an issue with photos and a map pin, follow its progress, upvote others' reports, and comment.
2. **Authority dashboard** — triage incoming reports, assign them to field staff, set deadlines, and close them with proof of work.
3. **Public map and transparency page** — anyone can see open and resolved issues and each area's response times.
4. **Admin panel** — manage departments, categories, areas and users.

Time allowed: 10–14 days. **Level:** mid to senior MERN developer (2+ years). The project tests data modeling, geospatial queries, real-time updates, authentication and authorization, file uploads, aggregation, testing and deployment — not just CRUD.

User roles and user stories

The system has five roles; every API route and screen must enforce them.

Role	Who	Can do
Guest	Any visitor	View public map, issue details and area stats

Role	Who	Can do
Citizen	Registered resident	Report, upvote, comment, follow issues, rate resolutions
Field Officer	Municipal worker	See assigned issues, update status, upload proof photos
Department Manager	Head of a department (e.g. Water, Roads)	Triage, assign, set priority and deadline, reject or merge duplicates
Admin	System owner	Manage users, departments, categories, areas and SLA rules

Core user stories

- As a **citizen**, I can report an issue with a title, description, category, up to 5 photos and a map location, so the right department receives it.
- As a **citizen**, I am warned when a similar open issue already exists within 100 m, so I can upvote it instead of creating a duplicate.
- As a **citizen**, I get a notification each time my issue changes status, and I can rate the fix (1–5) once it is resolved.
- As a **citizen**, I can reopen a resolved issue within 7 days if it was not actually fixed.
- As a **department manager**, I see a queue sorted by priority score and can assign an issue to a field officer with a deadline.
- As a **field officer**, I see my assigned issues on a map, and I must upload at least one "after" photo to mark an issue resolved.
- As a **manager**, I am alerted when an issue breaches its SLA deadline.
- As a **guest**, I can filter the public map by category, status, area and date, and see each area's average resolution time.
- As an **admin**, I can create departments, map categories to departments, and draw area boundaries.

Functional requirements

All items below are mandatory unless listed under Bonus challenges.

1. Authentication and accounts

- Sign up with email and password; email verification by link or 6-digit OTP.
- Login returns a short-lived access token (15 min) and a refresh token (7 days) in an httpOnly cookie, with refresh-token rotation.
- Forgot/reset password with an expiring token.
- Role-based access control on both backend and frontend; staff accounts are created only by an Admin.
- Profile page with name, avatar, phone and home area.

2. Issue reporting

- Form fields: title, description, category, location (pin on map + reverse-geocoded address), up to 5 images (max 5 MB each, jpg/png/webp), optional anonymous flag.
- Location stored as GeoJSON Point; the system auto-detects the Area the point falls in.
- **Duplicate detection:** before submit, query open issues of the same category within 100 m and show them to the user.
- Issue gets a readable ticket number, e.g. `CP-2026-000123`.

3. Issue lifecycle

Statuses and allowed transitions must be enforced on the server; any other transition returns 409.

```
stateDiagram-v2
    [*] --> Submitted
    Submitted --> Verified
    Submitted --> Rejected
    Submitted --> Duplicate
    Verified --> Assigned
    Assigned --> InProgress
    InProgress --> Resolved
    Resolved --> Closed
    Resolved --> Reopened
    Reopened --> Assigned
```

Every transition is written to an audit timeline: who, when, old status, new status, note.

4. Priority and SLA

- Each category has an SLA in hours (e.g. open manhole 24 h, garbage 72 h).
- Priority score = category severity × 10 + upvotes + hours open ÷ 6, recalculated on change.
- A scheduled job (every 15 min) flags issues past their deadline as `slaBreached` and notifies the manager.

5. Community engagement

- Upvote (one per user per issue), follow/unfollow, threaded comments (one level of replies).
- Report abusive comments; managers can hide them.
- Resolution rating (1–5) and feedback by the reporter.

6. Real-time notifications

- Socket.IO rooms per user and per issue.
- Live events: status change, new comment, assignment, SLA breach.
- In-app notification center with unread count and mark-as-read; notifications also stored in the database.

7. Map and search

- Interactive map (Leaflet or Mapbox) with marker clustering and color by status.
- Load issues only for the visible map bounds (geospatial `$geoWithin` query).
- Text search over title and description, plus filters: category, status, area, date range, "near me" radius.
- Server-side pagination (cursor or page-based) on all lists.

8. Dashboards and analytics

- **Manager dashboard:** open vs resolved counts, SLA breach rate, average resolution time, officer workload.
- **Public transparency page:** per-area table of issues reported, resolved and median resolution time, plus a 12-month trend chart.
- All figures computed with MongoDB aggregation pipelines, not in JavaScript loops.
- Export any report as CSV.

9. Admin panel

- CRUD for departments, categories (with SLA and severity), areas (GeoJSON polygons) and staff users.
- Deactivate users; activity log of admin actions.

Technical requirements and architecture

The stack is fixed to MERN with TypeScript; supporting libraries are the candidate's choice unless marked required.

Layer	Required	Candidate may choose
Frontend	React 18+, TypeScript, React Router	Redux Toolkit / RTK Query or TanStack Query; Tailwind or MUI; React Hook Form + Zod
Backend	Node.js 20+, Express, TypeScript	Layered or modular structure (routes → controllers → services → repositories)
Database	MongoDB 7 with Mongoose	Atlas or local Docker
Real-time	Socket.IO with JWT auth on handshake	Redis adapter (bonus)
File storage	Cloud storage, never the server disk	Cloudinary or AWS S3 (signed uploads)
Jobs	A scheduler for SLA checks	node-cron or BullMQ
Maps	Leaflet or Mapbox GL	Nominatim or Mapbox for geocoding
Validation	Server-side schema validation on every request	Zod or Joi
DevOps	Docker Compose for local run, `.env.example`	GitHub Actions for CI

```
flowchart LR
    A[React SPA] -->|REST + JWT| B[Express API]
    A <-->|Socket.IO| C[Realtime gateway]
    B --> D[(MongoDB)]
    B --> E[Cloudinary / S3]
    C --> D
    F[SLA scheduler] --> D
    F --> C
```

The React app talks to the API over REST and receives live events over Socket.IO; the scheduler writes SLA breaches and pushes them through the same socket gateway.

Required repository layout: a monorepo with `/client`, `/server` and `docker-compose.yml` at the root, or two repos with matching READMEs. Use a centralized error handler, a consistent response shape `{ success, data, error, meta }`, and environment config loaded and validated at startup.

Data model

Nine collections are the minimum; candidates may add more but must justify embedding vs referencing in their README.

Collection	Key fields	Required indexes
users	name, email, passwordHash, role, department, area, avatarUrl, isVerified, isActive	unique email
departments	name, description, managerId	unique name
categories	name, departmentId, severity (1–5), slaHours, icon	unique name
areas	name, boundary (GeoJSON Polygon)	2dsphere on boundary

Collection	Key fields	Required indexes
issues	ticketNo, title, description, categoryId, areaId, location (GeoJSON Point), address, images[], status, priorityScore, reporterId, isAnonymous, assignedTo, deadline, slaBreached, upvoteCount, resolution { note, images[], resolvedAt, rating, feedback }	2dsphere on location; text on title + description; compound on status + areaId + createdAt
issueEvents	issueId, actorId, fromStatus, toStatus, note, createdAt	issueId + createdAt
comments	issueId, authorId, parentId, body, isHidden, reports[]	issueId + createdAt
votes / follows	userId, issueId, type	unique userId + issueId + type
notifications	userId, type, issueId, message, readAt	userId + readAt

Rules to respect:

- Upvote counts are kept consistent with the votes collection (atomic `$inc` or a transaction).
- Status change and its `issueEvents` record are written in one MongoDB transaction.
- Passwords hashed with bcrypt or argon2; `passwordHash` never returned by the API.
- A seed script creates 3 departments, 8 categories, 4 areas, one user per role and at least 200 issues spread across areas and 6 months.

API specification

All routes are prefixed with `/api/v1`, return the standard response shape, and are documented in Swagger/OpenAPI at `/api/docs`.

Method	Endpoint	Access	Purpose
POST	/auth/register	Public	Create citizen account, send verification

Method	Endpoint	Access	Purpose
POST	/auth/verify-email	Public	Verify email token or OTP
POST	/auth/login	Public	Issue access + refresh tokens
POST	/auth/refresh	Cookie	Rotate refresh token
POST	/auth/logout	Auth	Revoke refresh token
POST	/auth/forgot-password, /auth/reset-password	Public	Password reset flow
GET	/issues	Public	List with filters, text search, pagination
GET	/issues/map?bbox=...	Public	Issues inside map bounds, light payload
GET	/issues/nearby? lng&lat&radius&category	Citizen	Duplicate check before submit
POST	/issues	Citizen	Create issue (multipart or pre-signed URLs)
GET	/issues/:id	Public	Issue detail with timeline
PATCH	/issues/:id/status	Manager, Officer	Status transition with note
PATCH	/issues/:id/assign	Manager	Assign officer and deadline
POST	/issues/:id/merge	Manager	Mark as duplicate of another issue
POST	/issues/:id/resolve	Officer	Resolve with proof images
POST	/issues/:id/reopen	Reporter	Reopen within 7 days
POST	/issues/:id/rate	Reporter	Rate the resolution
POST / DELETE	/issues/:id/vote, /issues/:id/follow	Citizen	Toggle upvote / follow

Method	Endpoint	Access	Purpose
GET / POST	/issues/:id/comments	Public / Citizen	List and add comments
PATCH	/comments/:id/hide	Manager	Hide abusive comment
GET	/notifications, PATCH /notifications/read	Auth	Notification center
GET	/stats/areas, /stats/trends	Public	Transparency figures
GET	/stats/dashboard, /stats/export.csv	Manager, Admin	Dashboard and CSV export
CRUD	/admin/users, /departments, /categories, /areas	Admin	Master data management

Socket events (server → client): `issue:statusChanged`, `issue:commented`, `issue:assigned`, `issue:slaBreached`, `notification:new`.

Non-functional requirements

These separate a senior submission from a working demo, and carry 35% of the score.

Security

- Helmet, strict CORS allow-list, and rate limiting (e.g. 5 login attempts per 15 min per IP, 20 issue reports per day per user).
- Input validation and sanitization against NoSQL injection and XSS on every route.
- File uploads checked by MIME type and size on the server, not only in the browser.
- Ownership checks: a citizen can edit or delete only their own issue while it is still `Submitted`.
- No secrets in the repository; `.env.example` lists every variable.

Performance

- Map endpoint returns at most 500 markers with only id, status, category and coordinates.
- List endpoints respond in under 300 ms on the seeded dataset (show `explain()` output for the two heaviest queries).

- Images resized/compressed before storage; thumbnails used in lists.
- Frontend: route-level code splitting, debounced search, optimistic UI for upvotes.

Code quality

- TypeScript strict mode on both sides; ESLint + Prettier with zero errors.
- No business logic in route handlers or React components — services and custom hooks.
- Meaningful commit history (not one final commit).

Testing

- Backend: Jest + Supertest integration tests covering auth, the status state machine, RBAC, and duplicate detection; minimum 70% line coverage on services.
- Use `mongodb-memory-server` or a test container, never the dev database.
- Frontend: React Testing Library tests for the report form and the notification center.
- At least one end-to-end flow (Playwright or Cypress): citizen reports → manager assigns → officer resolves → citizen rates.

Accessibility and UX

- Responsive from 360 px mobile to desktop; citizens will mostly report from phones.
- Keyboard navigable forms, labels on all inputs, sufficient color contrast.
- Loading, empty and error states on every data view.

Bonus challenges

Optional; each adds up to 2 points, capped at 10 bonus points total.

Bonus	What it proves
Offline reporting as a PWA: save a draft without network and sync when back online	Service workers, IndexedDB, sync logic
Redis caching for public stats plus the Socket.IO Redis adapter for horizontal scaling	Caching strategy and scaling
BullMQ queue for image processing and notifications instead of inline work	Background jobs and resilience

Bonus	What it proves
Heatmap layer showing issue density per area over time	Advanced geospatial aggregation
Bilingual UI (English + one local language, including RTL if relevant)	i18n architecture
Email or WhatsApp/SMS notification channel with user preferences	Third-party integration
Auto-suggest category from the description text	Applied NLP or an AI API, with a fallback
Live deployment (e.g. Render/Railway + Vercel + Atlas) with CI running tests on every push	Production readiness

Deliverables, timeline and submission

The candidate submits one repository link and a demo video by the end of day 14.

Deliverables checklist

- ☐ Source code in a GitHub repository (public, or private with the reviewer invited)
- ☐ `README.md` with setup steps, architecture overview, design decisions and known limitations
- ☐ `docker-compose up` starts client, server and MongoDB with seeded data in one command
- ☐ Seed script and demo credentials for every role
- ☐ Swagger/OpenAPI docs reachable at `/api/docs`
- ☐ Test suite runnable with one command, with the coverage report
- ☐ Postman collection or `.http` file for key flows
- ☐ 5–10 minute screen-recorded demo walking through the full issue lifecycle across all roles
- ☐ Live URL (optional, counts toward bonus)

Suggested timeline

Days	Milestone
1–2	Data model, project setup, auth with refresh tokens and RBAC
3–5	Issue reporting, image upload, geospatial queries, duplicate detection
6–7	Status lifecycle, assignment, SLA scheduler, audit timeline
8–9	Socket.IO notifications, comments, votes, follows
10–11	Map UI, dashboards, aggregation stats, CSV export, admin panel
12–13	Tests, security hardening, Docker, documentation
14	Demo video and submission

Rules: candidates may use libraries, documentation and AI assistants, but must be able to explain every part of the code in a 45-minute follow-up review, where they will also be asked to make one small live change.

Evaluation rubric

The submission is scored out of 100, plus up to 10 bonus points; 70+ is a pass and 85+ is a strong hire signal.

Area	Points	What reviewers check
Core features work end to end	25	All four portals, full lifecycle, duplicate detection, SLA flagging
Backend architecture and API design	15	Layering, validation, error handling, consistent responses, OpenAPI docs
Data modeling and MongoDB usage	15	Schema choices, indexes, geospatial queries, aggregations, transactions
Security and auth	10	Token rotation, RBAC on every route, rate limits, upload checks

Area	Points	What reviewers check
Frontend quality and UX	10	State management, forms, map performance, responsive design, loading/error states
Real-time features	5	Authenticated sockets, correct rooms, no duplicate listeners
Testing	10	Coverage, meaningful integration tests, one E2E flow
DevOps and documentation	10	Docker one-command run, seed data, README, commit history
Bonus	+10	See Bonus challenges

Automatic fail conditions: secrets committed to the repo, app does not start with the documented steps, or passwords stored in plain text.

Follow-up review (45 min): walkthrough of architecture, questions on two chosen files, one live change (e.g. add a new status or a new filter), and a short discussion of how the system would scale to one million issues.